

瑞吉外卖-第六天

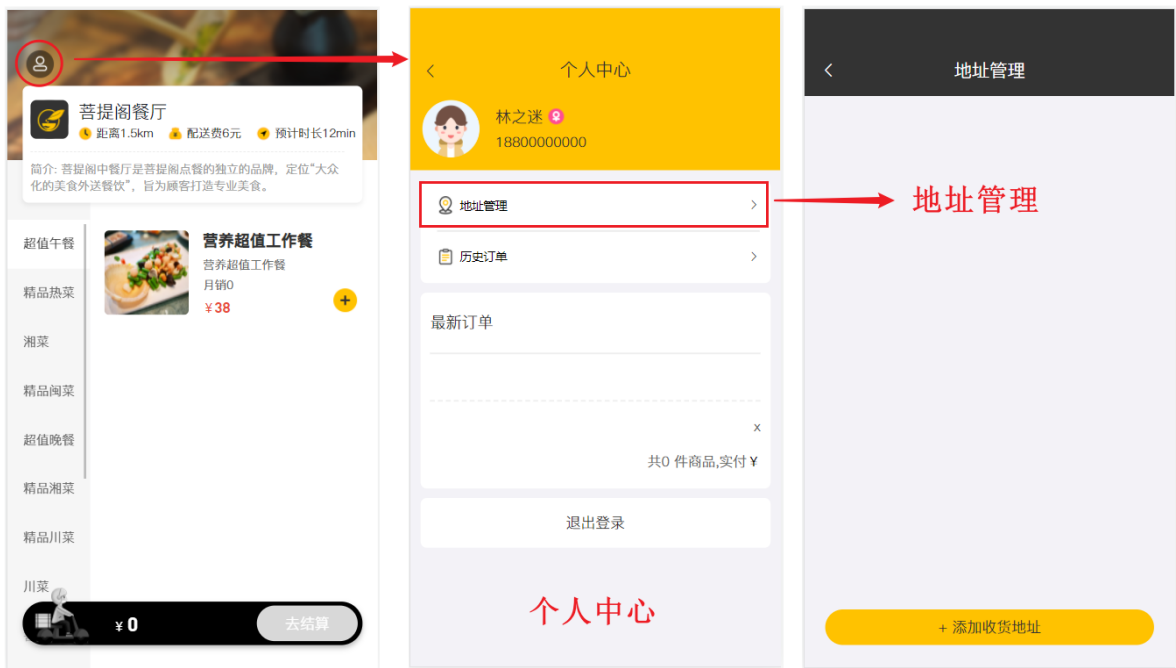
课程内容

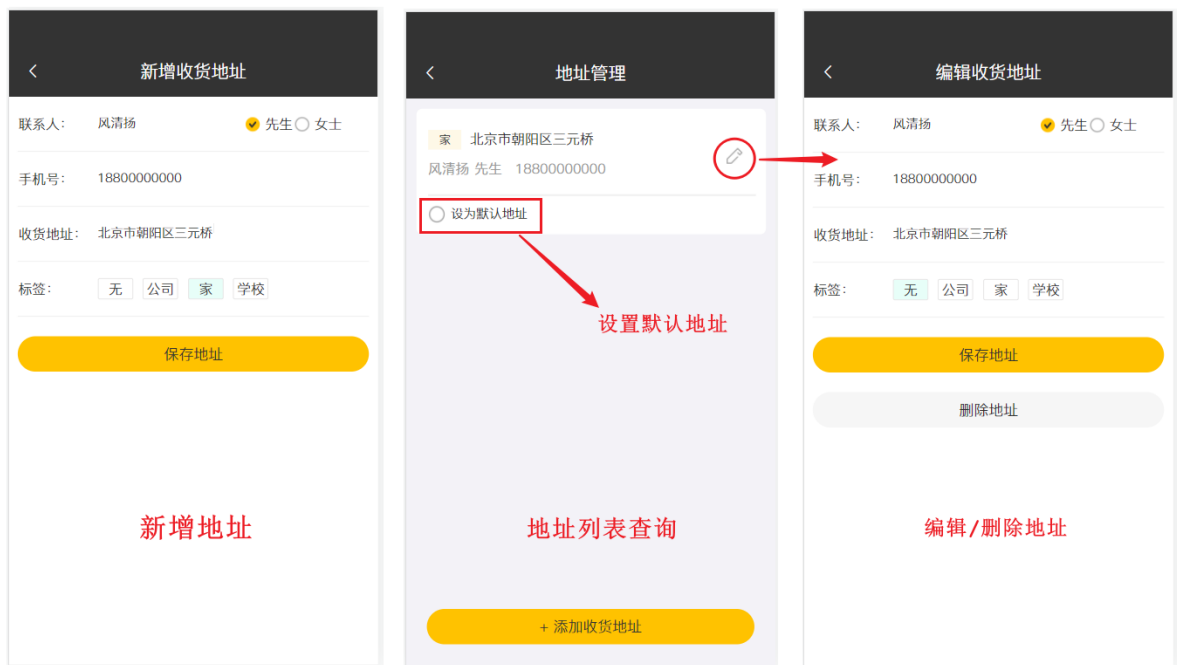
- 用户地址簿功能
- 菜品展示
- 购物车
- 下单

1. 用户地址簿功能

1.1 需求分析

地址簿，指的是移动端消费者用户的地址信息，用户登录成功后可以维护自己的地址信息。同一个用户可以有多个地址信息，但是只能有一个**默认地址**。





对于地址簿管理，我们需要实现以下几个功能：

- 新增地址
- 地址列表查询
- 设置默认地址
- 编辑地址
- 删除地址

1.2 数据模型

用户的地址信息会存储在address_book表，即地址簿表中。具体表结构如下：

名	类型	长度	小数点	不是 null	键	注释
id	bigint	20		☒	 1	主键
user_id	bigint	20		☒		用户id
consignee	varchar	50		☒		收货人
sex	tinyint	4		☒		性别 0 女 1 男
phone	varchar	11		☒		手机号
province_code	varchar	12		☐		省级区划编号
province_name	varchar	32		☐		省级名称
city_code	varchar	12		☐		市级区划编号
city_name	varchar	32		☐		市级名称
district_code	varchar	12		☐		区级区划编号
district_name	varchar	32		☐		区级名称
detail	varchar	200		☐		详细地址
label	varchar	100		☐		标签
is_default	tinyint	1		☒		默认 0 否 1是
create_time	datetime			☒		创建时间
update_time	datetime			☒		更新时间
create_user	bigint	20		☒		创建人
update_user	bigint	20		☒		修改人
is_deleted	int	11		☒		是否删除

这里面有一个字段is_default，实际上我们在设置默认地址时，只需要更新这个字段就可以了。

1.3 导入功能代码

对于这一类的单表的增删改查，我们已经写过很多了，基本的开发思路都是一样的，那么本小节的用户地址簿管理的增删改查功能，我们就不再一一实现了，基本的代码我们都已经提供了，直接导入进来，做一个测试即可。

对于下面的地址管理的代码，我们可以直接从资料拷贝，也可以直接从下面的讲义中复制。

1). 实体类 AddressBook (直接从课程资料中导入即可)

所属包: com.itheima.reggie.entity

```
1  import com.baomidou.mybatisplus.annotation.FieldFill;
2  import com.baomidou.mybatisplus.annotation.TableField;
3  import lombok.Data;
4  import java.io.Serializable;
5  import java.time.LocalDateTime;
6  /**
7   * 地址簿
8   */
9  @Data
10 public class AddressBook implements Serializable {
11     private static final long serialVersionUID = 1L;
12
13     private Long id;
14
15     //用户id
16     private Long userId;
17
18     //收货人
19     private String consignee;
20
21     //手机号
22     private String phone;
23
24     //性别 0 女 1 男
25     private String sex;
26
27     //省级区划编号
```

```
28     private String provinceCode;
29
30     //省级名称
31     private String provinceName;
32
33     //市级区划编号
34     private String cityCode;
35
36     //市级名称
37     private String cityName;
38
39     //区级区划编号
40     private String districtCode;
41
42     //区级名称
43     private String districtName;
44
45     //详细地址
46     private String detail;
47
48     //标签
49     private String label;
50
51     //是否默认 0 否 1是
52     private Integer isDefault;
53
54     //创建时间
55     @TableField(fill = FieldFill.INSERT)
56     private LocalDateTime createTime;
57
58     //更新时间
59     @TableField(fill = FieldFill.INSERT_UPDATE)
60     private LocalDateTime updateTime;
61
62     //创建人
63     @TableField(fill = FieldFill.INSERT)
64     private Long createUser;
```

```

65
66     //修改人
67     @TableField(fill = FieldFill.INSERT_UPDATE)
68     private Long updateUser;
69
70     //是否删除
71     private Integer isDeleted;
72 }

```

2) . Mapper接口 AddressBookMapper (直接从课程资料中导入即可)

所属包: com.itheima.reggie.mapper

```

1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.reggie.entity.AddressBook;
3  import org.apache.ibatis.annotations.Mapper;
4  import org.apache.ibatis.annotations.Select;
5  import java.util.List;
6
7  @Mapper
8  public interface AddressBookMapper extends BaseMapper<AddressBook> {
9  }

```

3) . 业务层接口 AddressBookService (直接从课程资料中导入即可)

所属包: com.itheima.reggie.service

```

1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.reggie.entity.AddressBook;
3
4  public interface AddressBookService extends IService<AddressBook> {
5  }

```

4) . 业务层实现类 AddressBookServiceImpl (直接从课程资料中导入即可)

所属包: com.itheima.reggie.service.impl

```

1  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
2  import com.itheima.reggie.entity.AddressBook;
3  import com.itheima.reggie.mapper.AddressBookMapper;
4  import com.itheima.reggie.service.AddressBookService;
5  import org.springframework.stereotype.Service;
6
7  @Service
8  public class AddressBookServiceImpl extends ServiceImpl<AddressBookMapper,
    AddressBook> implements AddressBookService {
9  }

```

5). 控制层 AddressBookController (直接从课程资料中导入即可)

所属包: com.itheima.reggie.controller

controller主要开发的功能:

A. 新增地址逻辑说明:

- 需要记录当前是哪个用户的地址 (关联当前登录用户)

B. 设置默认地址

- 每个用户可以有很多地址, 但是默认地址只能有一个 ;
- 先将该用户所有地址的is_default更新为0 , 然后将当前的设置的默认地址的is_default设置为1

C. 根据ID查询地址

D. 查询默认地址

- 根据当前登录用户ID 以及 is_default进行查询, 查询当前登录用户is_default为1的地址信息

E. 查询指定用户的全部地址

- 根据当前登录用户ID, 查询所有的地址列表

代码实现如下:

```

1  import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;

```

```
2  import com.baomidou.mybatisplus.core.conditions.update.LambdaUpdateWrapper;
3  import com.itheima.reggie.common.BaseContext;
4  import com.itheima.reggie.common.R;
5  import com.itheima.reggie.entity.AddressBook;
6  import com.itheima.reggie.service.AddressBookService;
7  import lombok.extern.slf4j.Slf4j;
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.web.bind.annotation.*;
10 import java.util.List;
11
12 /**
13  * 地址簿管理
14  */
15 @Slf4j
16 @RestController
17 @RequestMapping("/addressBook")
18 public class AddressBookController {
19     @Autowired
20     private AddressBookService addressBookService;
21
22     /**
23      * 新增
24      */
25     @PostMapping
26     public R<AddressBook> save(@RequestBody AddressBook addressBook) {
27         addressBook.setUserId(BaseContext.getCurrentId());
28         log.info("addressBook:{}", addressBook);
29         addressBookService.save(addressBook);
30         return R.success(addressBook);
31     }
32
33     /**
34      * 设置默认地址
35      */
36     @PutMapping("default")
37     public R<AddressBook> setDefault(@RequestBody AddressBook addressBook) {
38         log.info("addressBook:{}", addressBook);
```

```

39         LambdaUpdateWrapper<AddressBook> wrapper = new LambdaUpdateWrapper<>();
40         wrapper.eq(AddressBook::getUserId, BaseContext.getCurrentId());
41         wrapper.set(AddressBook::getIsDefault, 0);
42         //SQL:update address_book set is_default = 0 where user_id = ?
43         addressBookService.update(wrapper);
44
45         addressBook.setIsDefault(1);
46         //SQL:update address_book set is_default = 1 where id = ?
47         addressBookService.updateById(addressBook);
48         return R.success(addressBook);
49     }
50
51     /**
52      * 根据id查询地址
53      */
54     @GetMapping("/{id}")
55     public R get(@PathVariable Long id) {
56         AddressBook addressBook = addressBookService.getById(id);
57         if (addressBook != null) {
58             return R.success(addressBook);
59         } else {
60             return R.error("没有找到该对象");
61         }
62     }
63
64     /**
65      * 查询默认地址
66      */
67     @GetMapping("default")
68     public R<AddressBook> getDefault() {
69         LambdaQueryWrapper<AddressBook> queryWrapper = new LambdaQueryWrapper<>
70         ();
71         queryWrapper.eq(AddressBook::getUserId, BaseContext.getCurrentId());
72         queryWrapper.eq(AddressBook::getIsDefault, 1);
73
74         //SQL:select * from address_book where user_id = ? and is_default = 1
75         AddressBook addressBook = addressBookService.getOne(queryWrapper);

```



```

75
76         if (null == addressBook) {
77             return R.error("没有找到该对象");
78         } else {
79             return R.success(addressBook);
80         }
81     }
82
83     /**
84      * 查询指定用户的全部地址
85      */
86     @GetMapping("/list")
87     public R<List<AddressBook>> list(AddressBook addressBook) {
88         addressBook.setUserId(BaseContext.getCurrentId());
89         log.info("addressBook:{}", addressBook);
90
91         //条件构造器
92         LambdaQueryWrapper<AddressBook> queryWrapper = new LambdaQueryWrapper<>
93             ();
94         queryWrapper.eq(null != addressBook.getUserId(), AddressBook::getUserId,
95             addressBook.getUserId());
96         queryWrapper.orderByDesc(AddressBook::getUpdateTime);
97
98         //SQL:select * from address_book where user_id = ? order by update_time
99         desc
100
101         return R.success(addressBookService.list(queryWrapper));
102     }
103 }

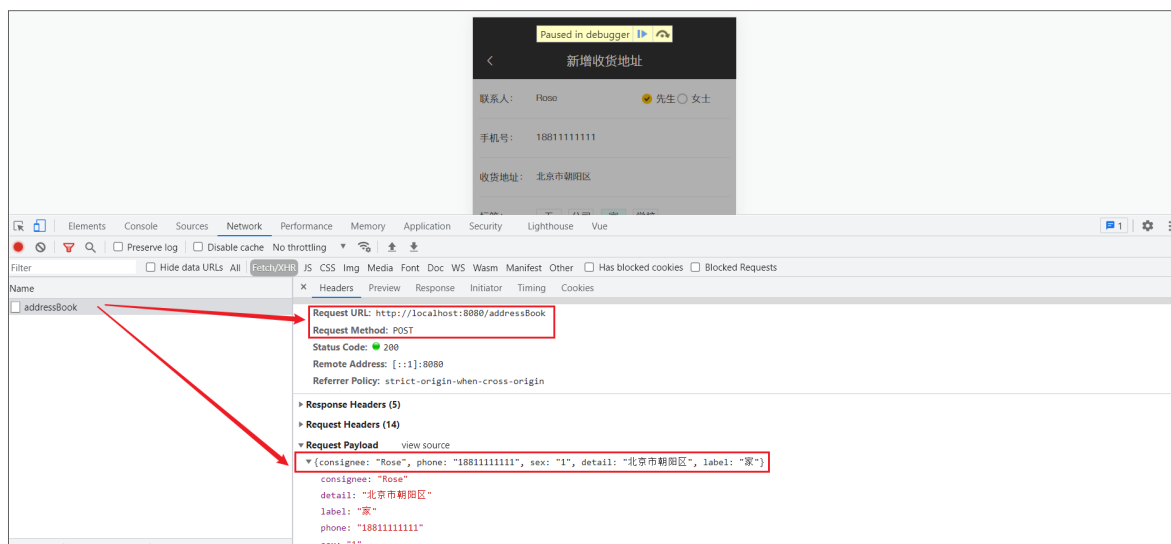
```

1.4 功能测试

代码导入进来，并且去阅读了一下地址管理各个功能的逻辑实现，接下来，我们就可以启动项目，进行一个测试。测试过程中，通过debug断点调试观察服务端程序的执行过程，在浏览器中使用调试工具查看页面和服务端的交互过程和请求响应数据。

1). 新增

填写表单数据，点击保存地址，查看网络请求。

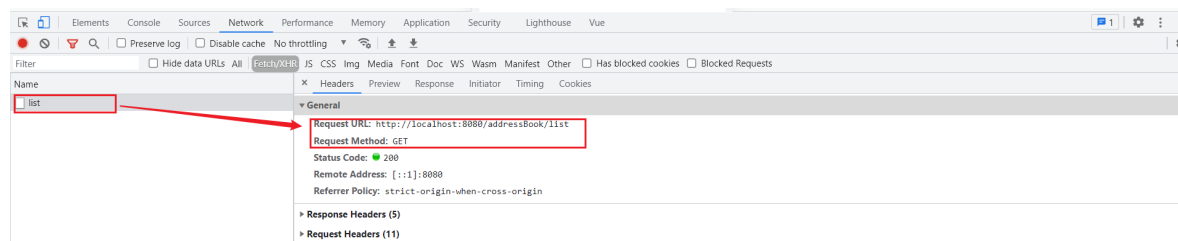


测试完毕之后，检查数据库中的数据，是否正常插入。

id	user_id	consignee	sex	phone	province_code	province_name	city_code	city_name	district_code
1425792459560005634	1425754006214488065	风清扬	1	188000000000	(Null)	(Null)	(Null)	(Null)	(Null)
1425792881775423489	1425754006214488065	Rose	1	188111111111	(Null)	(Null)	(Null)	(Null)	(Null)

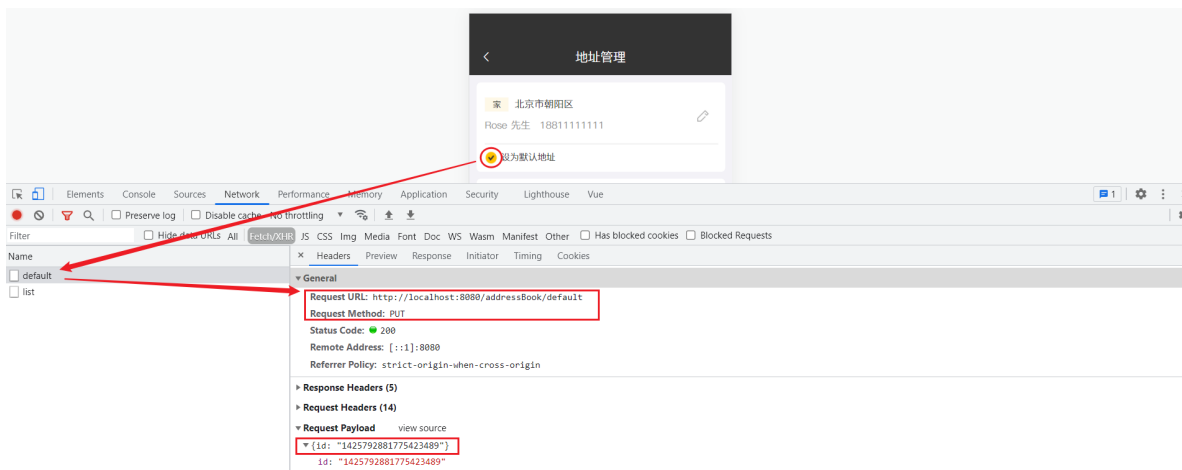
2). 列表查询

当新增地址完成后，页面会再次发送一个请求，来查询该用户的所有地址列表，在界面进行展示。



3). 设置默认

在地址列表页面中，勾选 "设为默认地址"，此时会发送PUT请求，来设置默认地址。



测试完毕后，我们再次查看数据库表中的数据：

detail	label	is_default	create_time	update_time	create_user	update_user
北京市朝阳区三元桥	家	0	2021-08-12 20:13:06	2021-08-12 20:26:15	1425754006214488065	1425754006214488065
北京市朝阳区	家	1	2021-08-12 20:14:47	2021-08-12 20:27:04	1425754006214488065	1425754006214488065

2. 菜品展示

2.1 需求分析

用户登录成功后跳转到系统首页，在首页需要根据分类来展示菜品和套餐。如果菜品设置了口味信息，需要展示 **选择规格** 按钮，否则显示 **+** 按钮。



2.2 前端页面分析

在开发代码之前，需要梳理一下前端页面和服务端的交互过程：

1). 页面(front/index.html)发送ajax请求，获取分类数据（菜品分类和套餐分类）

```
mounted() {
  this.initData()
},
methods: {
  // 初始化数据
  initData() {
    Promise.all([categoryListApi(), cartListApi({})]).then(res => {
      // 获取分类数据
      if (res[0].code === 1) {
        this.categoryList = res[0].data
        if (Array.isArray(res[0].data) && res[0].data.length > 0) {
          this.categoryId = res[0].data[0].id
          if (res[0].data[0].type === 1) {
            this.getDishList()
          } else {
            this.getSetmealData()
          }
        }
      } else {
        this.$notify({ type: 'warning', message: res[0].msg })
      }
    })
  }
}

// 获取所有的菜品分类
function categoryListApi() {
  return $axios({
    'url': '/category/list',
    'method': 'get',
  })
}
```

页面加载时调用

发送两个请求，分别获取分类列表，购物车列表
两个都执行成功，才能正常渲染页面

该功能在之前的业务开发中，我们都已经实现了。通过请求响应的数据，我们也可以看到数据是可以正确获取到的。

加载分类列表，执行成功

获取购物车列表，还未实现

Name	Status	Type	Initiator	Size	Time	Waterfall
list	200	xhr	axios.min.js2	2.9 kB	47 ms	
list	404	xhr	axios.min.js2	274 B	13 ms	

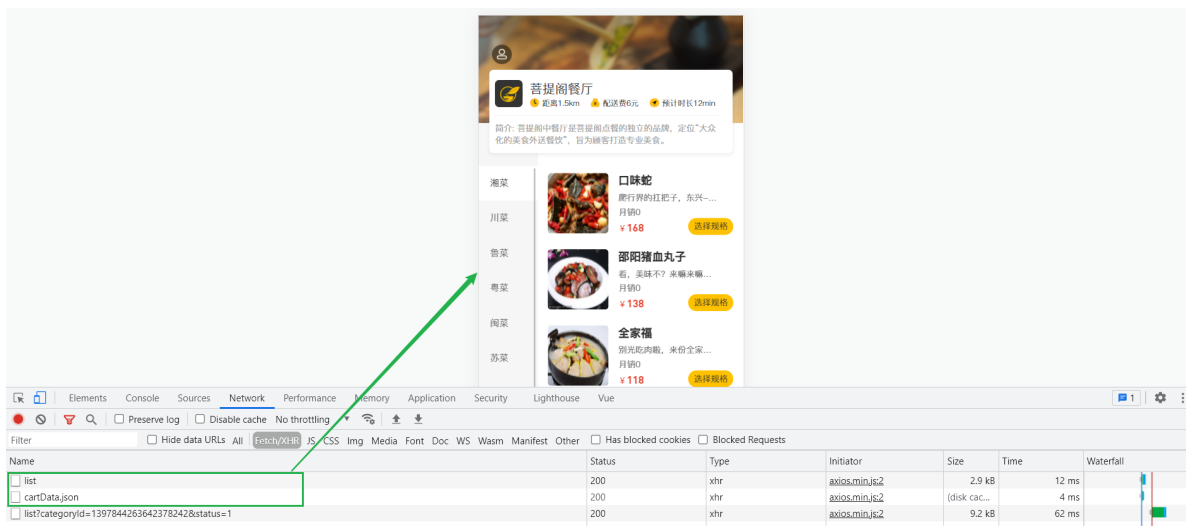
注意：首页加载时，不仅发送请求获取分类列表，还发送了一次ajax请求用于加载购物车数据，而这两次请求必须全部成功，页面才可以正常渲染，而当前购物车列表查询功能还未实现（报404），所以列表目前并未渲染。此处可以将这次请求的地址暂时修改一下，从静态json文件获取数据，等后续开发购物车功能时再修改回来，如下：

```
// 获取购物车内商品的集合
function cartListApi(data) {
  return $axios({
    // 'url': '/shoppingCart/list',
    'url': '/front/cartData.json',
    'method': 'get',
    params: {...data}
  })
}
```

cartData.json

```
1 [{"code":1,"msg":null,"data":[],"map":{}}]
```

修改之后，我们再次测试：



目前该部分的功能我们已经调通，左侧的分类菜单，和右侧的菜品信息我们都可以看到，后续我们只需要将购物车列表的数据改成调用服务端接口查询即可。

2). 页面发送ajax请求，获取第一个分类下的菜品或者套餐

```
initData(){
    // 查询分类信息 查询购物车列表
    Promise.all([categoryListApi(), cartListApi({})]).then(res=>{
        // 获取分类数据
        if(res[0].code === 1){
            this.categoryList = res[0].data
            if(Array.isArray(res[0].data) && res[0].data.length > 0){
                this.categoryId = res[0].data[0].id // 获取到列表的第一个分类ID
                if(res[0].data[0].type === 1){
                    this.getDishList() // 分类type为1, 代表菜品, 查询对应菜品列表
                }else{
                    this.getSetmealData() // 分类type为2, 代表套餐, 查询对应套餐列表
                }
            }
        }else{
            this.$notify({ type:'warning', message:res[0].msg});
        }
    })
}
```

A. 根据分类ID查询套餐列表：

```
// 获取套餐数据setmealId
async getSetmealData(){
    if(!this.categoryId){
        return
    }
    const res = await setmealListApi({categoryId:this.categoryId,status:1}) // 根据分类ID, 查询套餐列表
    if(res.code === 1){
        let dishList = res.data
        const cartData = this.cartData
        if(dishList.length > 0 && cartData.length > 0){
            dishList.forEach(dish=>{
                cartData.forEach(cart=>{
                    if(dish.id === cart.setmealId){
                        dish.number = cart.number
                    }
                })
            })
        }
        this.dishList = dishList
    }
}

// 获取菜品分类对应的套餐
function setmealListApi(data) {
    return $axios({
        'url': '/setmeal/list',
        'method': 'get',
        'params': {...data}
    })
}
```

B. 根据分类ID查询菜品列表：

```
async getDishList() {
  if (!this.categoryId) {
    return
  }
  const res = await dishListApi({categoryId:this.categoryId,status:1})
  if (res.code === 1) {
    let dishList = res.data
    const cartData = this.cartData
    if (dishList.length > 0 && cartData.length > 0) {
      dishList.forEach(dish=>{
        cartData.forEach(cart=>{
          if (dish.id === cart.dishId) {
            dish.number = cart.number
          }
        })
      })
    }
    this.dishList = dishList
  }
}
```

// 获取菜品分类对应的菜品

```
function dishListApi(data) {
  return $axios({
    'url': '/dish/list',
    'method': 'get',
    'params': {...data}
  })
}
```

根据分类ID, 查询菜品列表

异步请求，查询分类对应的菜品列表，功能我们已经实现了，但是我们之前查询的只是菜品的基本信息，不包含菜品的口味信息。所以在前端界面中，我们看不到选择菜品分类的信息。

```
@GetMapping("/list")
public R<List<Dish>> list(Dish dish) {
  // 构造查询条件
  LambdaQueryWrapper<Dish> queryWrapper = new LambdaQueryWrapper<>();
  queryWrapper.eq( condition: dish.getCategoryId() != null ,Dish::getCategoryId,dish.getCategoryId());
  // 添加条件， 查询状态为1（起售状态）的菜品
  queryWrapper.eq(Dish::getStatus, val: 1);

  // 添加排序条件
  queryWrapper.orderByAsc(Dish::getSort).orderByDesc(Dish::getUpdateTime);

  List<Dish> list = dishService.list(queryWrapper);

  return R.success(list);
}
```

之前实现的根据分类查询菜品列表，仅查询了菜品基本信息，未包含菜品口味信息

经过上述的分析，我们可以看到，服务端我们主要提供两个方法， 分别用来：

A. 根据分类ID查询菜品列表 (包含菜品口味列表)， 具体请求信息如下：

请求	说明
请求方式	GET
请求路径	/dish/list
请求参数	?categoryId=1397844263642378242&status=1

该功能在服务端已经实现，我们需要修改此方法，在原有方法的基础上增加查询菜品的口味信息。

B. 根据分类ID查询套餐列表， 具体请求信息如下：

请求	说明
请求方式	GET
请求路径	/setmeal/list
请求参数	?categoryId=1397844263642378242&status=1

该功能在服务端并未实现。

2.3 代码开发

2.3.1 查询菜品方法修改

由于之前我们实现的根据分类查询菜品列表，仅仅查询了菜品的基本信息，未查询菜品口味信息，而移动端用户在点餐时，是需要选择口味信息的，所以我们需要对之前的代码实现进行完善，那么如何完善呢？

我们需要修改DishController的list方法，原来此方法的返回值类型为：R<List>。为了满足移动端对数据的要求（菜品基本信息和菜品对应的口味信息），现在需要将方法的返回值类型改为：

R<List>，因为在DishDto中封装了菜品对应的口味信息：

```
@Data
public class DishDto extends Dish {

    //菜品对应的口味数据
    private List<DishFlavor> flavors = new ArrayList<>();

    private String categoryName;

    private Integer copies;
}
```

代码逻辑：

- 根据分类ID查询，查询目前正在启售的菜品列表（已实现）
- 遍历菜品列表，并查询菜品的分类信息及菜品的口味列表
- 组装数据DishDto，并返回

代码实现：

```
1 @GetMapping("/list")
```

```
2     public R<List<DishDto>> list(Dish dish){
3         //构造查询条件
4         LambdaQueryWrapper<Dish> queryWrapper = new LambdaQueryWrapper<>();
5         queryWrapper.eq(dish.getCategoryId() != null
6             ,Dish::getCategoryId,dish.getCategoryId());
7         //添加条件，查询状态为1（起售状态）的菜品
8         queryWrapper.eq(Dish::getStatus,1);
9         //添加排序条件
10        queryWrapper.orderByAsc(Dish::getSort).orderByDesc(Dish::getUpdateTime);
11
12        List<Dish> list = dishService.list(queryWrapper);
13
14        List<DishDto> dishDtoList = list.stream().map((item) -> {
15            DishDto dishDto = new DishDto();
16            BeanUtils.copyProperties(item,dishDto);
17
18            Long categoryId = item.getCategoryId();//分类id
19            //根据id查询分类对象
20            Category category = categoryService.getById(categoryId);
21            if(category != null){
22                String categoryName = category.getName();
23                dishDto.setCategoryName(categoryName);
24            }
25
26            //当前菜品的id
27            Long dishId = item.getId();
28            LambdaQueryWrapper<DishFlavor> lambdaQueryWrapper = new
29            LambdaQueryWrapper<>();
30            lambdaQueryWrapper.eq(DishFlavor::getDishId,dishId);
31            //SQL:select * from dish_flavor where dish_id = ?
32            List<DishFlavor> dishFlavorList =
33            dishFlavorService.list(lambdaQueryWrapper);
34            dishDto.setFlavors(dishFlavorList);
35
36            return dishDto;
37        }).collect(Collectors.toList());
```



```
36         return R.success(dishDtoList);
37     }
```

2.3.2 根据分类ID查询套餐

在SetmealController中创建list方法，根据条件查询套餐数据。

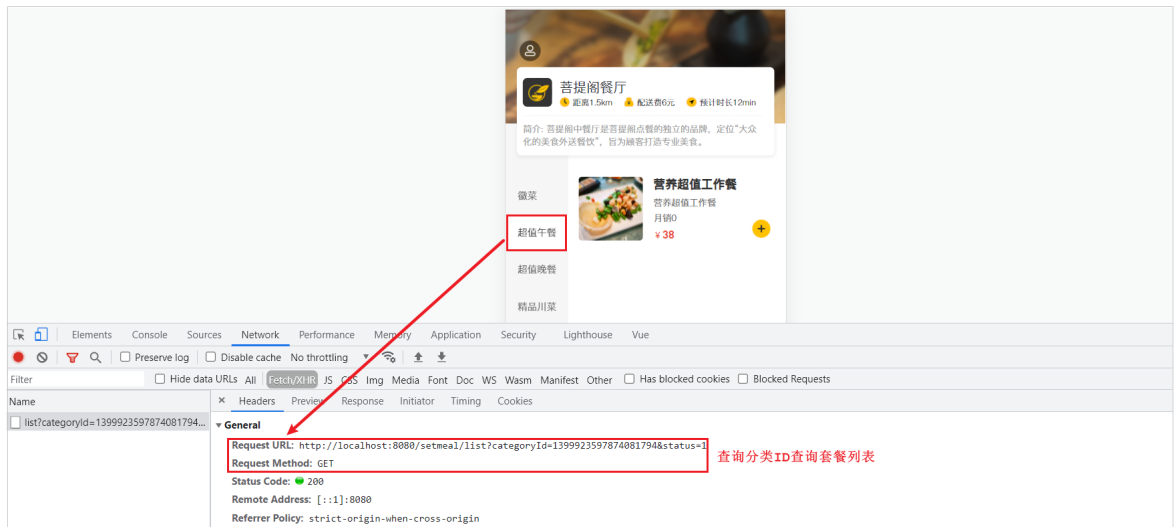
```
1  /**
2   * 根据条件查询套餐数据
3   * @param setmeal
4   * @return
5   */
6  @GetMapping("/list")
7  public R<List<Setmeal>> list(Setmeal setmeal) {
8      LambdaQueryWrapper<Setmeal> queryWrapper = new LambdaQueryWrapper<>();
9      queryWrapper.eq(setmeal.getCategoryId() !=
10         null, Setmeal::getCategoryId, setmeal.getCategoryId());
11      queryWrapper.eq(setmeal.getStatus() !=
12         null, Setmeal::getStatus, setmeal.getStatus());
13      queryWrapper.orderByDesc(Setmeal::getUpdateTime);
14
15      List<Setmeal> list = setmealService.list(queryWrapper);
16
17      return R.success(list);
18  }
```

2.4 功能测试

把菜品展示的功能代码完善完成之后，我们重新启动服务，来测试一个菜品展示的功能。测试过程中可以使用浏览器的监控工具查看页面和服务端的数据交互细节。



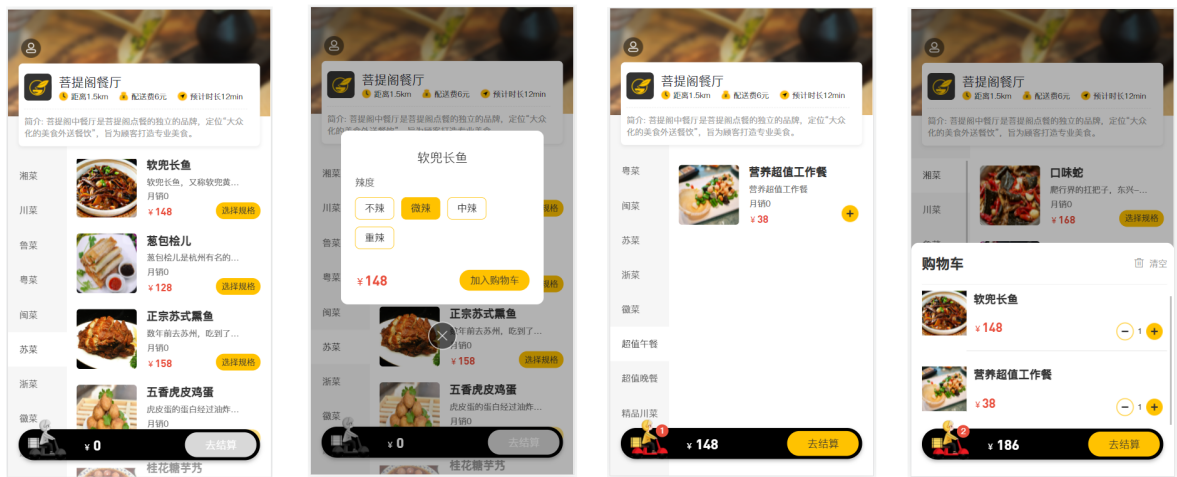
点击分类，根据分类查询菜品列表/套餐列表：



3. 购物车

3.1 需求分析

移动端用户可以将菜品或者套餐添加到购物车。对于菜品来说，如果设置了口味信息，则需要选择规格后才能加入购物车；对于套餐来说，可以直接点击 $\oplus$ 将当前套餐加入购物车。在购物车中可以修改菜品和套餐的数量，也可以清空购物车。



这里面我们需要实现的功能包括：

- 1) 添加购物车
- 2) 查询购物车
- 3) 清空购物车

3.2 数据模型

用户的购物车数据，也是需要保存在数据库中的，购物车对应的数据表为shopping_cart表，具体表结构如下：

名	类型	长度	小数点	不是 null	键	注释
id	bigint	20		☒	1	主键
name	varchar	50		☐		名称
image	varchar	100		☐		图片
user_id	bigint	20		☒		主键
dish_id	bigint	20		☐		菜品id
setmeal_id	bigint	20		☐		套餐id
dish_flavor	varchar	50		☐		口味
number	int	11		☒		数量
amount	decimal	10	2	☒		金额
create_time	datetime			☐		创建时间

说明：

- 购物车数据是关联用户的，在表结构中，我们需要记录，每一个用户的购物车数据是哪些
- 菜品列表展示出来的既有套餐，又有菜品，如果APP端选择的是套餐，就保存套餐ID(setmeal_id)，如果APP端选择的是菜品，就保存菜品ID(dish_id)
- 对同一个菜品/套餐，如果选择多份不需要添加多条记录，增加数量number即可

最终shopping_cart表中存储的数据示例：

id	name	image	user_id	dish_id	setmeal_id	dish_flavor	number	amount
1426363041544470530	软兜长鱼	7d522623-8	1425754006214488065	1398092095142014978	(Null)	微辣	1	148.00
1426363051388502017	葱包桧儿	311a3a0e-d	1425754006214488065	1398091889449152513	(Null)	微辣	1	128.00
1426363067339440129	米饭	44e0e090-b	1425754006214488065	1423329009705463809	(Null)	(Null)	1	2.00
1426363077372215298	雪碧	f5657d6d-a	1425754006214488065	1423328235936067586	(Null)	(Null)	1	5.00
1426853966381178881	佛跳墙	da9e1c70-f	1426853398040403969	1422783914845487106	(Null)	微辣, 不要葱	1	888.00
1426853980117524482	米饭	44e0e090-b	1426853398040403969	1423329009705463809	(Null)	(Null)	1	2.00
1426853991941267457	雪碧	f5657d6d-a	1426853398040403969	1423328235936067586	(Null)	(Null)	1	5.00
1426853995388985346	可乐	b5282101-f	1426853398040403969	1423328152549109762	(Null)	(Null)	1	5.00
1426854157981179906	营养超值二	9cd7a80a-c	1426853398040403969	(Null)	1423329486060957698	(Null)	1	38.00

通过user_id区分是哪个用户的购物车

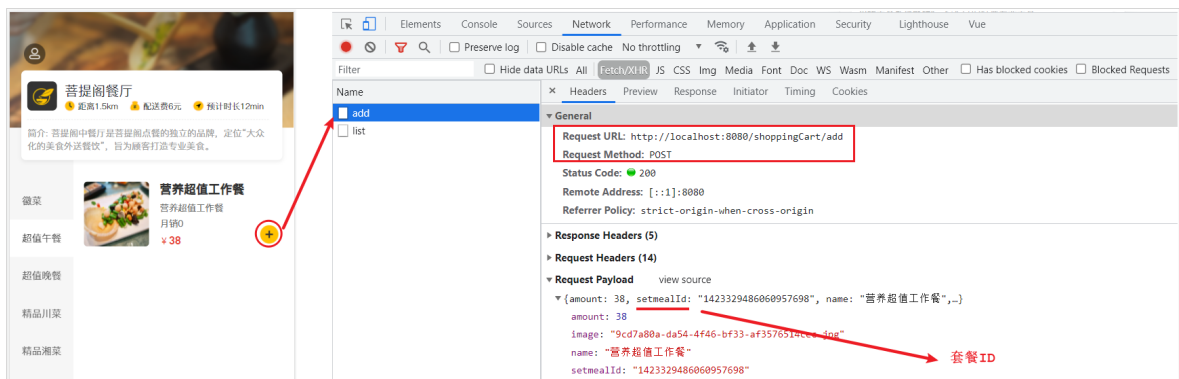
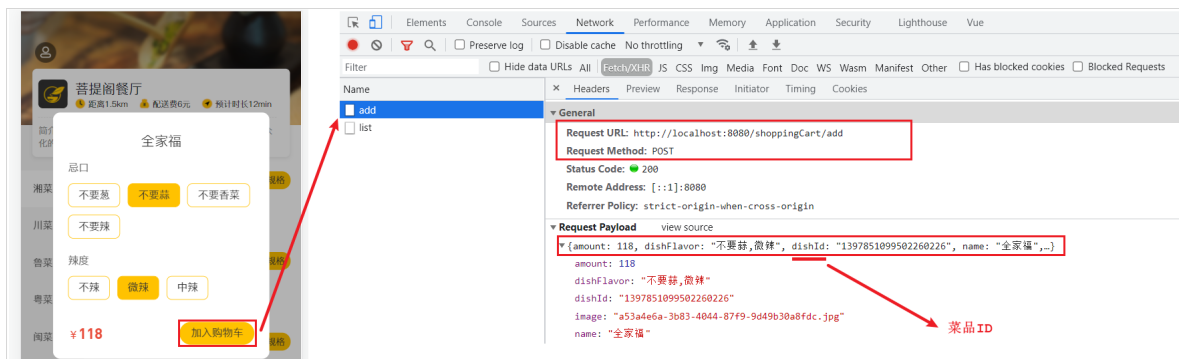
加入购物车的菜品

加入购物车的套餐

3.3 前端页面分析

在开发代码之前，需要梳理一下购物车操作时前端页面和服务端的交互过程：

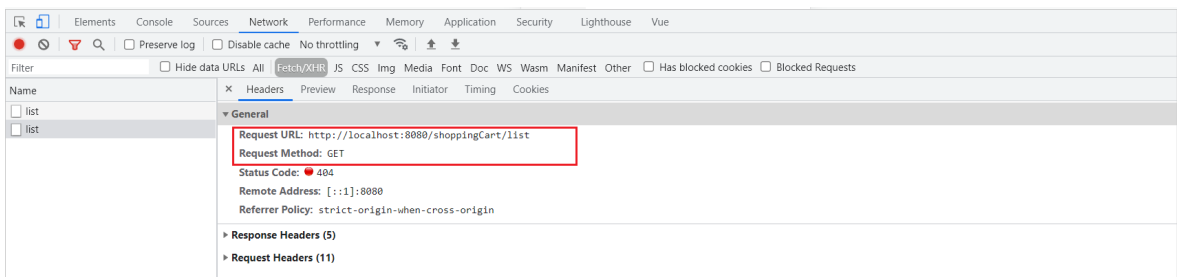
1). 点击 "加入购物车" 或者 "+" 按钮，页面发送ajax请求，请求服务端，将菜品或者套餐添加到购物车



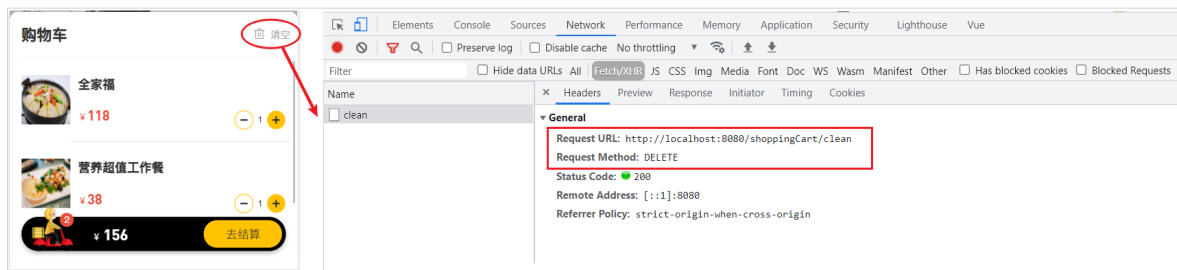
2). 点击购物车图标，页面发送ajax请求，请求服务端查询购物车中的菜品和套餐

此时，我们就需要将查询购物车的代码放开，不用再加载静态的json数据了。

```
//获取购物车内商品的集合
function cartListApi(data) {
  return $axios({
    'url': '/shoppingCart/list',
    // 'url': '/front/cartData.json',
    'method': 'get',
    params: {...data}
  })
}
```



3) . 点击清空购物车按钮，页面发送ajax请求，请求服务端来执行清空购物车操作



经过上述的分析，我们可以看到，对于购物车的功能，我们主要需要开发以下几个功能，具体的请求信息如下：

1) . 加入购物车

请求	说明
请求方式	POST
请求路径	/shoppingCart/add
请求参数	json格式

```
1  菜品数据：
2  {"amount":118,"dishFlavor":"不要蒜,微辣","dishId":"1397851099502260226","name":"全家福","image":"a53a4e6a-3b83-4044-87f9-9d49b30a8fdc.jpg"}
3
4  套餐数据：
5  {"amount":38,"setmealId":"1423329486060957698","name":"营养超值工作餐","image":"9cd7a80a-da54-4f46-bf33-af3576514cec.jpg"}
```

2). 查询购物车列表

请求	说明
请求方式	GET
请求路径	/shoppingCart/list

3). 清空购物车功能

请求	说明
请求方式	DELETE
请求路径	/shoppingCart/clean

3.4 准备工作

分析完毕购物车的业务需求和实现思路之后，在开发业务功能前，先将需要用到的类和接口基本结构创建好：

1). 实体类 ShoppingCart (直接从课程资料中导入即可)

所属包：com.itheima.reggie.entity

```
1  import lombok.Data;
2  import java.io.Serializable;
3  import java.math.BigDecimal;
4  import java.time.LocalDateTime;
```

```
5
6  /**
7   * 购物车
8   */
9  @Data
10 public class ShoppingCart implements Serializable {
11     private static final long serialVersionUID = 1L;
12
13     private Long id;
14
15     //名称
16     private String name;
17
18     //用户id
19     private Long userId;
20
21     //菜品id
22     private Long dishId;
23
24     //套餐id
25     private Long setmealId;
26
27     //口味
28     private String dishFlavor;
29
30     //数量
31     private Integer number;
32
33     //金额
34     private BigDecimal amount;
35
36     //图片
37     private String image;
38
39     private LocalDateTime createTime;
40 }
```

2). Mapper接口 ShoppingCartMapper

所属包: com.itheima.reggie.mapper

```
1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.reggie.entity.ShoppingCart;
3  import org.apache.ibatis.annotations.Mapper;
4
5  @Mapper
6  public interface ShoppingCartMapper extends BaseMapper<ShoppingCart> {
7  }
```

3). 业务层接口 ShoppingCartService

所属包: com.itheima.reggie.service

```
1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.reggie.entity.ShoppingCart;
3
4  public interface ShoppingCartService extends IService<ShoppingCart> {
5  }
```

4). 业务层实现类 ShoppingCartServiceImpl

所属包: com.itheima.reggie.service.impl

```
1  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
2  import com.itheima.reggie.entity.ShoppingCart;
3  import com.itheima.reggie.mapper.ShoppingCartMapper;
4  import com.itheima.reggie.service.ShoppingCartService;
5  import org.springframework.stereotype.Service;
6
7  @Service
8  public class ShoppingCartServiceImpl extends ServiceImpl<ShoppingCartMapper,
9      ShoppingCart> implements ShoppingCartService {
9  }
```


5). 控制层 ShoppingCartController

所属包: com.itheima.reggie.controller

```
1  import com.itheima.reggie.service.ShoppingCartService;
2  import lombok.extern.slf4j.Slf4j;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.*;
5
6  /**
7   * 购物车
8   */
9  @Slf4j
10 @RestController
11 @RequestMapping("/shoppingCart")
12 public class ShoppingCartController {
13     @Autowired
14     private ShoppingCartService shoppingCartService;
15
16 }
```

3.5 代码开发

3.5.1 添加购物车

在ShoppingCartController中创建add方法，来完成添加购物车的逻辑实现，具体的逻辑如下：

- A. 获取当前登录用户，为购物车对象赋值
- B. 根据当前登录用户ID 及 本次添加的菜品ID/套餐ID，查询购物车数据是否存在
- C. 如果已经存在，就在原来数量基础上加1
- D. 如果不存在，则添加到购物车，数量默认就是1

代码实现如下：

```
1  /**
```

```
2    * 添加购物车
3    * @param shoppingCart
4    * @return
5    */
6    @PostMapping("/add")
7    public R<ShoppingCart> add(@RequestBody ShoppingCart shoppingCart){
8        log.info("购物车数据:{}", shoppingCart);
9
10       //设置用户id, 指定当前是哪个用户的购物车数据
11       Long currentId = BaseContext.getCurrentId();
12       shoppingCart.setUserId(currentId);
13
14       Long dishId = shoppingCart.getDishId();
15
16       LambdaQueryWrapper<ShoppingCart> queryWrapper = new LambdaQueryWrapper<>();
17       queryWrapper.eq(ShoppingCart::getUserId, currentId);
18
19       if(dishId != null){
20           //添加到购物车的是菜品
21           queryWrapper.eq(ShoppingCart::getDishId, dishId);
22       }else{
23           //添加到购物车的是套餐
24           queryWrapper.eq(ShoppingCart::getSetmealId, shoppingCart.getSetmealId());
25       }
26
27       //查询当前菜品或者套餐是否在购物车中
28       //SQL:select * from shopping_cart where user_id = ? and dish_id/setmeal_id =
29       ?
29       ShoppingCart cartServiceOne = shoppingCartService.getOne(queryWrapper);
30
31       if(cartServiceOne != null){
32           //如果已经存在, 就在原来数量基础上加一
33           Integer number = cartServiceOne.getNumber();
34           cartServiceOne.setNumber(number + 1);
35           shoppingCartService.updateById(cartServiceOne);
36       }else{
37           //如果不存在, 则添加到购物车, 数量默认就是一
```

```
38         shoppingCart.setNumber(1);
39         shoppingCart.setCreateTime(LocalDateTime.now());
40         shoppingCartService.save(shoppingCart);
41         cartServiceOne = shoppingCart;
42     }
43     return R.success(cartServiceOne);
44 }
```

3.5.2 查询购物车

在ShoppingCartController中创建list方法，根据当前登录用户ID查询购物车列表，并对查询的结果进行创建时间的倒序排序。

代码实现如下：

```
1  /**
2   * 查看购物车
3   * @return
4   */
5   @GetMapping("/list")
6   public R<List<ShoppingCart>> list() {
7       log.info("查看购物车...");
8
9       LambdaQueryWrapper<ShoppingCart> queryWrapper = new LambdaQueryWrapper<>();
10      queryWrapper.eq(ShoppingCart::getUserId,BaseContext.getCurrentId());
11      queryWrapper.orderByAsc(ShoppingCart::getCreateTime);
12
13      List<ShoppingCart> list = shoppingCartService.list(queryWrapper);
14
15      return R.success(list);
16  }
```

3.5.3 清空购物车

在ShoppingCartController中创建clean方法，在方法中获取当前登录用户，根据登录用户ID，删除购物车数据。

代码实现如下：

```
1  /**
2   * 清空购物车
3   * @return
4   */
5   @DeleteMapping("/clean")
6   public R<String> clean() {
7       //SQL:delete from shopping_cart where user_id = ?
8       LambdaQueryWrapper<ShoppingCart> queryWrapper = new LambdaQueryWrapper<>();
9       queryWrapper.eq(ShoppingCart::getUserId,BaseContext.getCurrentId());
10
11       shoppingCartService.remove(queryWrapper);
12       return R.success("清空购物车成功");
13   }
```

3.6 功能测试

按照前面分析的操作流程进行测试，测试功能以及数据库中的数据是否是否正常。

1) . 添加购物车

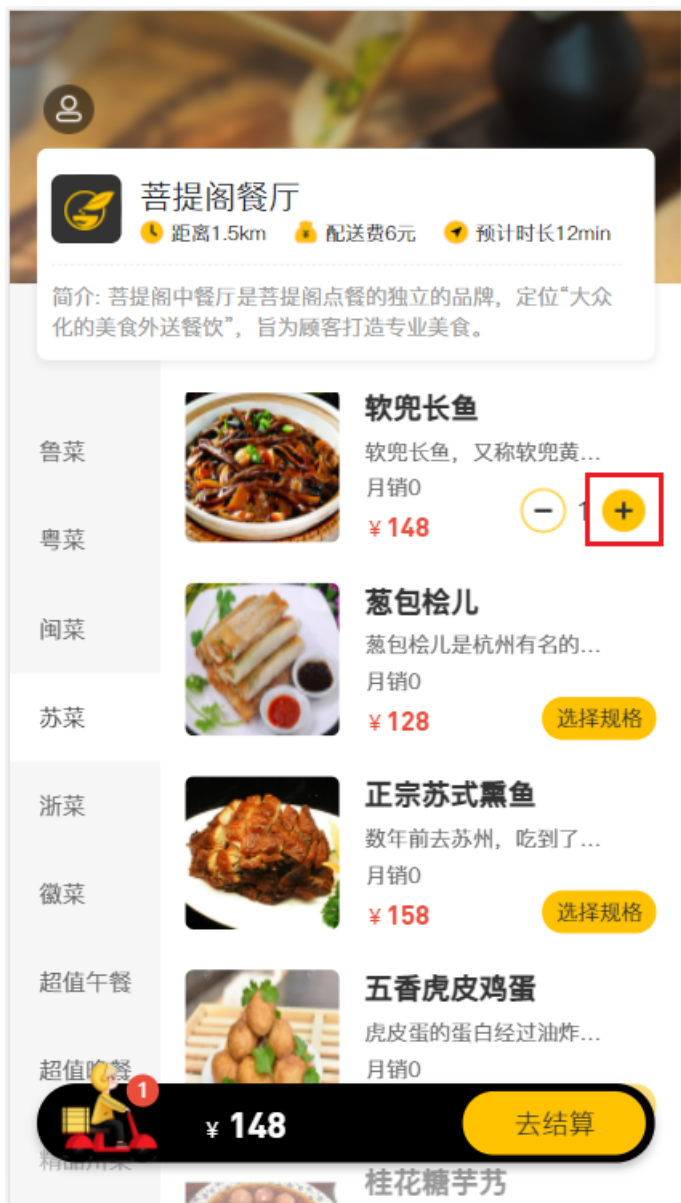
当添加的是菜品信息，而这个用户的购物车中当前并没有这个菜品时，添加一条数据，数量为1。



检查数据库数据，由于是菜品保存的是dish_id。

id	name	image	user_id	dish_id	setmeal_id	dish_flavor	number	amount	create_time
1426319451493539841	软兜长鱼	7d52262	1425754006214488065	1398092095142014978	(Null)	微辣	1	148.00	2021-08-14 07:07:11

这时在页面上，我们可以继续点击+号，在购物车中增加该菜品，此时，应该是对现有的购物车菜品数量加1，而不应该插入新的记录。



检查数据库数据:

id	name	image	user_id	dish_id	setmeal_id	dish_flavor	number	amount	create_time
1426321259494416385	软兜长鱼	7d522	1425754006214488065	1398092095142014978	(Null)	微辣	2	148.00	2021-08-14 07:14:22

如果添加的是套餐, 该套餐在当前用户的购物车中并不存在, 则添加一条数据, 数量为1。



检查数据库数据：

id	name	image	user_id	dish_id	setmeal_id	dish_flavor	number	amount
1426321259494416385	软兜长鱼	7d522	1425754006214488065	1398092095142014978	(Null)	微辣	2	148.00
1426322154034601986	营养超值工作餐	9cd7a	1425754006214488065	(Null)	1423329486060957698	(Null)	1	38.00

2). 查看购物车

点击页面下面的购物车边栏，查看购物车数据列表是否正常展示。



3). 清空购物车

在购物车列表展示页中点击"清空", 查看购物车是否被清空。



并检查数据库中的数据，可以看到数据已经被删除。

4. 下单

4.1 需求分析

移动端用户将菜品或者套餐加入购物车后，可以点击购物车中的“去结算”按钮，页面跳转到订单确认页面，点击“去支付”按钮则完成下单操作。



这里，我们需要说明一下，这里并不会去开发支付功能，因为不论是支付宝的支付，还是微信支付，都是需要企业资质的，而我们大家在测试的时候，是没有办法提供企业资质的，所以这一部分支付功能我们就不去实现了。

4.2 数据模型

用户下单业务对应的数据表为orders表和order_detail表(一对多关系,一个订单关联多个订单明细)：

表名	含义	说明
orders	订单表	主要存储订单的基本信息(如：订单号、状态、金额、支付方式、下单用户、收件地址等)
order_detail	订单明细表	主要存储订单详情信息(如：该订单关联的套餐及菜品的信息)

具体的表结构如下：

A. orders 订单表

名	类型	长度	小数点	不是 null	键	注释
id	bigint	20		☒	1	主键
number	varchar	50		☐		订单号
status	int	11		☒		订单状态 1待付款, 2待派送, 3已派送, 4已完成, 5已
user_id	bigint	20		☒		下单用户
address_book_id	bigint	20		☒		地址id
order_time	datetime			☒		下单时间
checkout_time	datetime			☒		结账时间
pay_method	int	11		☒		支付方式 1微信,2支付宝
amount	decimal	10	2	☒		实收金额
remark	varchar	100		☐		备注
phone	varchar	255		☐		手机号
address	varchar	255		☐		地址
user_name	varchar	255		☐		用户名
consignee	varchar	255		☐		收货人

B. order_detail

名	类型	长度	小数点	不是 null	键	注释
id	bigint	20		☒	1	主键
name	varchar	50		☐		名字
image	varchar	100		☐		图片
order_id	bigint	20		☒		订单id
dish_id	bigint	20		☐		菜品id
setmeal_id	bigint	20		☐		套餐id
dish_flavor	varchar	50		☐		口味
number	int	11		☒		数量
amount	decimal	10	2	☒		金额

数据示例：

id	number	status	user_id	address_book_id	order_time	checkout_time	pay_method	amount	remark
1426344746728620034	1426344746728620034	2	1425754006214488065	560005634	2021-08-14 01:20:21	2021-08-14 08:11	1	283.00	
1426853833350438913	1426853833350438913	2	1426853398040403969	959868929	2021-08-15 11:11		1	909.00	三幅餐具啥

订单表

一个订单对应多个订单明细

id	name	image	order_id	dish_id	setmeal_id	dish_flavor	number	amount
1426344746728620034	软兜长鱼	7d522623-8c9e-4	1426344746728620034	1398092095142014978	(Null)	微辣	1	148.00
1426344746728620034	葱包桧儿	311a330e-d44d-4	1426344746728620034	1398091889449152513	(Null)	微辣	1	128.00
1426344746770563073	米饭	44e0e090-b44e-4	1426344746728620034	1423329009705463809	(Null)	(Null)	1	2.00
1426344746770563074	雪碧	f5657d6d-a283-4	1426344746728620034	1423328235936067586	(Null)	(Null)	1	5.00
1426853833396576257	佛跳墙	da9e1c70-fc32-4	1426853833350438913	1422783914845487106	(Null)	微辣,不要蒜	1	888.00
1426853833404964865	米饭	44e0e090-b44e-4	1426853833350438913	1423329009705463809	(Null)	(Null)	3	2.00
1426853833404964866	可乐	b5282101-138b-4	1426853833350438913	1423328152549109762	(Null)	(Null)	1	5.00
1426853833413353473	雪碧	f5657d6d-a283-4	1426853833350438913	1423328235936067586	(Null)	(Null)	1	5.00
1426853833413353474	北冰洋	c6938683-31b0-4	1426853833350438913	1423328312909934594	(Null)	(Null)	1	5.00

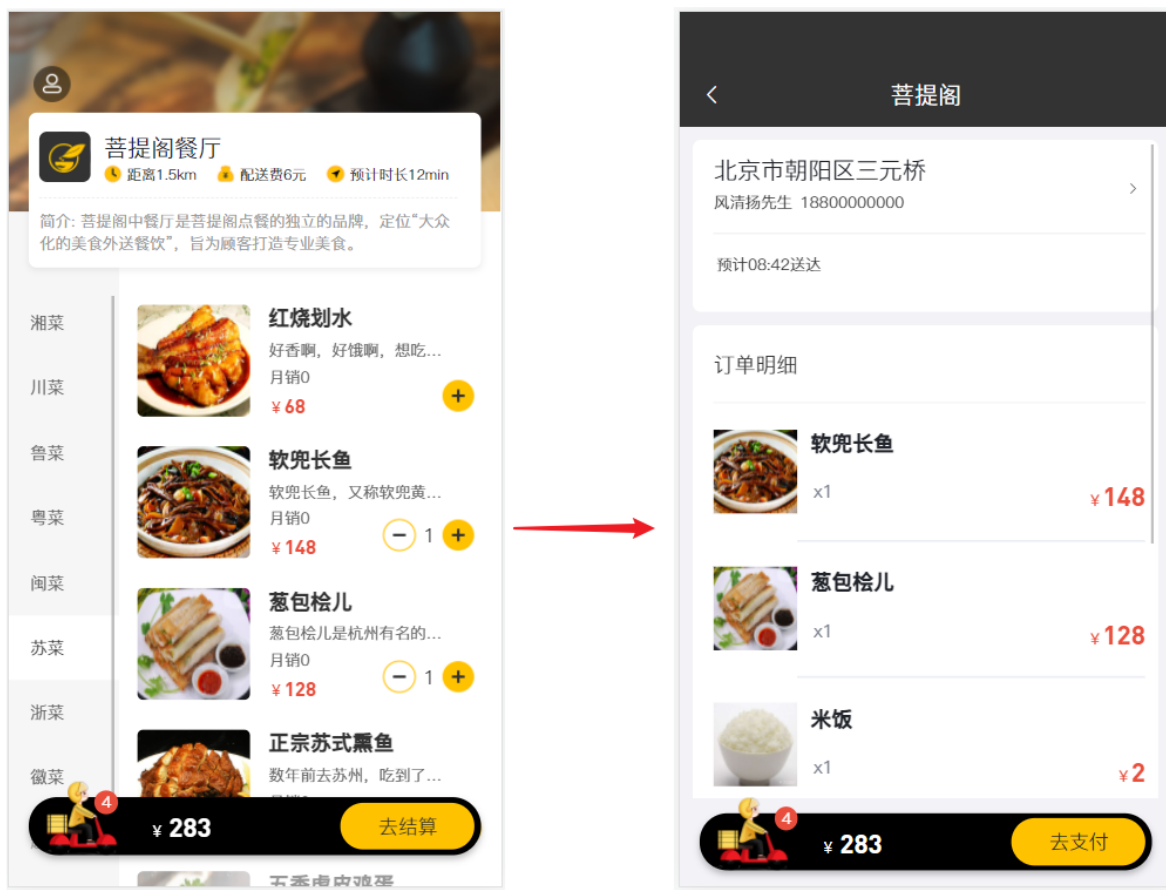
订单明细表

用户提交订单时，需要往订单表orders中插入一条记录，并且需要往order_detail中插入一条或多条记录。

4.3 前端页面分析

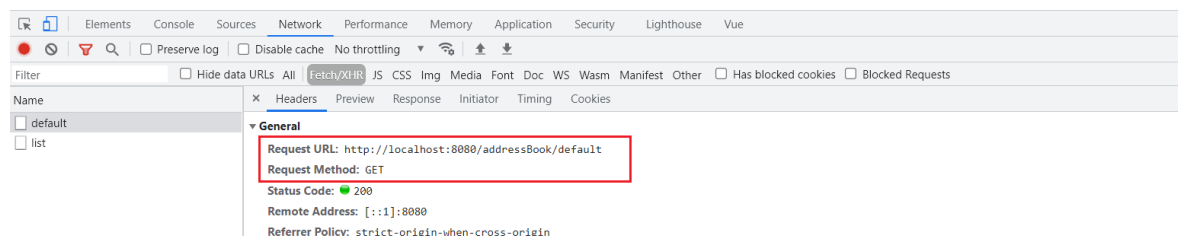
在开发代码之前，需要梳理一下用户下单操作时前端页面和服务端的交互过程：

- 1). 在购物车中点击  按钮，页面跳转到订单确认页面



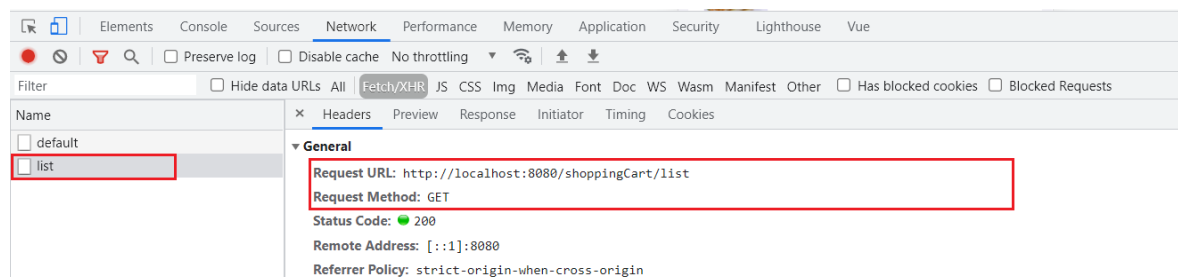
页面跳转前端已经完成，我们无需操作。

2) . 在订单确认页面，发送ajax请求，请求服务端获取当前登录用户的默认地址



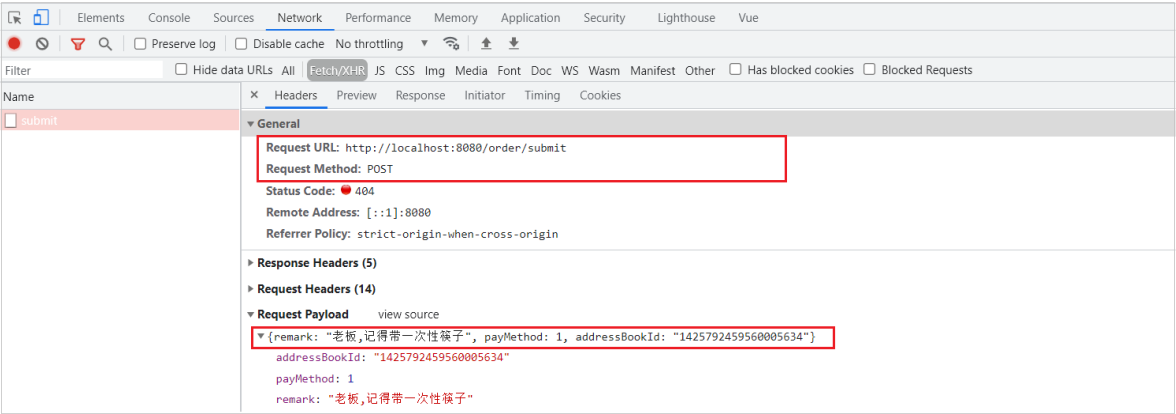
该功能在用户地址簿管理功能开发时，已经实现（导入），我们无需操作。

3) . 在订单确认页面，发送ajax请求，请求服务端获取当前登录用户的购物车数据



该功能已经实现，我们无需操作。

4). 在订单确认页面点击  按钮，发送ajax请求，请求服务端完成下单操作



经过上述的分析，我们看到前三步的功能我们都已经实现了，我们主要需要实现最后一步的下单功能，该功能具体的请求信息如下：

请求	说明
请求方式	POST
请求路径	/order/submit
请求参数	{ "remark": "老板,记得带一次性筷子", "payMethod": 1, "addressBookId": "1425792459560005634" }

4.4 准备工作

在开发业务功能前，先将需要用到的类和接口基本结构创建好：

1). 实体类 `Orders`、`OrderDetail`（直接从课程资料中导入即可）

所属包：`com.itheima.reggie.entity`

```
1  import lombok.Data;
2  import java.io.Serializable;
3  import java.math.BigDecimal;
4  import java.time.LocalDateTime;
5
6  /**
7   * 订单
```

```
8      */
9      @Data
10     public class Orders implements Serializable {
11         private static final long serialVersionUID = 1L;
12
13         private Long id;
14         //订单号
15         private String number;
16         //订单状态 1待付款, 2待派送, 3已派送, 4已完成, 5已取消
17         private Integer status;
18
19         //下单用户id
20         private Long userId;
21
22         //地址id
23         private Long addressBookId;
24
25         //下单时间
26         private LocalDateTime orderTime;
27
28         //结账时间
29         private LocalDateTime checkoutTime;
30
31         //支付方式 1微信, 2支付宝
32         private Integer payMethod;
33
34         //实收金额
35         private BigDecimal amount;
36
37         //备注
38         private String remark;
39
40         //用户名
41         private String userName;
42
43         //手机号
44         private String phone;
```

```
45
46     //地址
47     private String address;
48
49     //收货人
50     private String consignee;
51 }
```

```
1  import lombok.Data;
2  import java.io.Serializable;
3  import java.math.BigDecimal;
4
5  /**
6   * 订单明细
7   */
8  @Data
9  public class OrderDetail implements Serializable {
10     private static final long serialVersionUID = 1L;
11
12     private Long id;
13
14     //名称
15     private String name;
16
17     //订单id
18     private Long orderId;
19
20     //菜品id
21     private Long dishId;
22
23     //套餐id
24     private Long setmealId;
25
26     //口味
27     private String dishFlavor;
28 }
```

```

29      //数量
30      private Integer number;
31
32      //金额
33      private BigDecimal amount;
34
35      //图片
36      private String image;
37  }

```

2). Mapper接口 OrderMapper、OrderDetailMapper

所属包: com.itheima.reggie.mapper

```

1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.reggie.entity.Orders;
3  import org.apache.ibatis.annotations.Mapper;
4
5  @Mapper
6  public interface OrderMapper extends BaseMapper<Orders> {
7  }

```

```

1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.reggie.entity.OrderDetail;
3  import org.apache.ibatis.annotations.Mapper;
4
5  @Mapper
6  public interface OrderDetailMapper extends BaseMapper<OrderDetail> {
7  }

```

3). 业务层接口 OrderService、OrderDetailsService

所属包: com.itheima.reggie.service


```

1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.reggie.entity.Orders;
3
4  public interface OrderService extends IService<Orders> {
5  }

```

```

1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.reggie.entity.OrderDetail;
3
4  public interface OrderDetailService extends IService<OrderDetail> {
5  }

```

4). 业务层实现类 OrderServiceImpl、OrderDetailServiceImpl

所属包: com.itheima.reggie.service.impl

```

1  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
2  import com.itheima.reggie.entity.*;
3  import com.itheima.reggie.mapper.OrderMapper;
4  import com.itheima.reggie.service.*;
5  import lombok.extern.slf4j.Slf4j;
6  import org.springframework.stereotype.Service;
7
8  @Service
9  @Slf4j
10 public class OrderServiceImpl extends ServiceImpl<OrderMapper, Orders> implements
    OrderService {
11 }

```

```

1  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
2  import com.itheima.reggie.entity.OrderDetail;
3  import com.itheima.reggie.mapper.OrderDetailMapper;
4  import com.itheima.reggie.service.OrderDetailService;
5  import org.springframework.stereotype.Service;
6
7  @Service
8  public class OrderDetailServiceImpl extends ServiceImpl<OrderDetailMapper,
    OrderDetail> implements OrderDetailService {
9  }

```

5). 控制层 OrderController、OrderDetailController

所属包: com.itheima.reggie.controller

```

1  import com.itheima.reggie.service.OrderService;
2  import lombok.extern.slf4j.Slf4j;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.RequestMapping;
5  import org.springframework.web.bind.annotation.RestController;
6
7  /**
8   * 订单
9   */
10 @Slf4j
11 @RestController
12 @RequestMapping("/order")
13 public class OrderController {
14     @Autowired
15     private OrderService orderService;
16 }

```

```

1  import com.itheima.reggie.service.OrderDetailService;
2  import lombok.extern.slf4j.Slf4j;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.*;
5

```

```

6    /**
7     * 订单明细
8     */
9    @Slf4j
10   @RestController
11   @RequestMapping("/orderDetail")
12   public class OrderDetailController {
13
14       @Autowired
15       private OrderDetailService orderDetailService;
16   }

```

4.5 代码开发

在OrderController中创建submit方法，处理用户下单的逻辑：

```

1    /**
2     * 用户下单
3     * @param orders
4     * @return
5     */
6    @PostMapping("/submit")
7    public R<String> submit(@RequestBody Orders orders) {
8        log.info("订单数据: {}", orders);
9        orderService.submit(orders);
10        return R.success("下单成功");
11    }

```

由于下单的逻辑相对复杂，我们可以在OrderService中定义submit方法，来处理下单的具体逻辑：

```

1    /**
2     * 用户下单
3     * @param orders
4     */
5    public void submit(Orders orders);

```

然后在OrderServiceImpl中完成下单功能的具体实现，下单功能的具体逻辑如下：

- A. 获得当前用户id， 查询当前用户的购物车数据
- B. 根据当前登录用户id， 查询用户数据
- C. 根据地址ID， 查询地址数据
- D. 组装订单明细数据， 批量保存订单明细
- E. 组装订单数据， 批量保存订单数据
- F. 删除当前用户的购物车列表数据

具体代码实现如下：

```
1    @Autowired
2    private ShoppingCartService shoppingCartService;
3
4    @Autowired
5    private UserService userService;
6
7    @Autowired
8    private AddressBookService addressBookService;
9
10   @Autowired
11   private OrderDetailService orderDetailService;
12
13   /**
14    * 用户下单
15    * @param orders
16    */
17   @Transactional
18   public void submit(Orders orders) {
19       //获得当前用户id
20       Long userId = BaseContext.getCurrentId();
21
22       //查询当前用户的购物车数据
23       LambdaQueryWrapper<ShoppingCart> wrapper = new LambdaQueryWrapper<>();
24       wrapper.eq(ShoppingCart::getUserId,userId);
```

```
25     List<ShoppingCart> shoppingCarts = shoppingCartService.list(wrapper);
26
27     if(shoppingCarts == null || shoppingCarts.size() == 0){
28         throw new CustomException("购物车为空，不能下单");
29     }
30
31     //查询用户数据
32     User user = userService.getById(userId);
33
34     //查询地址数据
35     Long addressBookId = orders.getAddressBookId();
36     AddressBook addressBook = addressBookService.getById(addressBookId);
37     if(addressBook == null){
38         throw new CustomException("用户地址信息有误，不能下单");
39     }
40
41     long orderId = IdWorker.getId();//订单号
42
43     AtomicInteger amount = new AtomicInteger(0);
44
45     //组装订单明细信息
46     List<OrderDetail> orderDetails = shoppingCarts.stream().map((item) -> {
47         OrderDetail orderDetail = new OrderDetail();
48         orderDetail.setOrderId(orderId);
49         orderDetail.setNumber(item.getNumber());
50         orderDetail.setDishFlavor(item.getDishFlavor());
51         orderDetail.setDishId(item.getDishId());
52         orderDetail.setSetmealId(item.getSetmealId());
53         orderDetail.setName(item.getName());
54         orderDetail.setImage(item.getImage());
55         orderDetail.setAmount(item.getAmount());
56         amount.addAndGet(item.getAmount().multiply(new
BigDecimal(item.getNumber()).intValue()));
57         return orderDetail;
58     }).collect(Collectors.toList());
59
60     //组装订单数据
```

```

61     orders.setId(orderId);
62     orders.setOrderTime(LocalDateTime.now());
63     orders.setCheckoutTime(LocalDateTime.now());
64     orders.setStatus(2);
65     orders.setAmount(new BigDecimal(amount.get())); //总金额
66     orders.setUserId(userId);
67     orders.setNumber(String.valueOf(orderId));
68     orders.setUserName(user.getName());
69     orders.setConsignee(addressBook.getConsignee());
70     orders.setPhone(addressBook.getPhone());
71     orders.setAddress((addressBook.getProvinceName() == null ? "" :
addressBook.getProvinceName())
72         + (addressBook.getCityName() == null ? "" : addressBook.getCityName())
73         + (addressBook.getDistrictName() == null ? "" :
addressBook.getDistrictName())
74         + (addressBook.getDetail() == null ? "" : addressBook.getDetail()));
75     //向订单表插入数据，一条数据
76     this.save(orders);
77
78     //向订单明细表插入数据，多条数据
79     orderDetailService.saveBatch(orderDetails);
80
81     //清空购物车数据
82     shoppingCartService.remove(wrapper);
83 }

```

备注：

上述逻辑处理中，计算购物车商品的总金额时，为保证我们每一次执行的累加计算是一个原子操作，我们这里用到了JDK中提供的一个原子类 `AtomicInteger`

4.6 功能测试

代码编写完成，我们重新启动服务，按照前面分析的操作流程进行测试，查看数据是否正常即可。在测试过程中，我们可以通过debug的形式来跟踪代码的正常执行。



检查数据库数据

订单表插入一条记录:

id	number	status	user_id	address_book_order_time	checkout_time	pay_method	amount	remark
1426344746728620034	1426344746728620034		2125754006214488065	459560005634	2021-08-14 00:20:21	2021-08-14 08:4	1	283.00

订单明细表插入四条记录()：

id	name	image	order_id	dish_id	setmeal_id	dish_flavor	number	amount
1426344746762174466	软兜长鱼	7d522623-8c9e-4	1426344746728620034	1398092095142014978	(Null)	微辣	1	148.00
1426344746762174467	葱包桧儿	311a3a0e-d44d-4	1426344746728620034	139809188949152513	(Null)	微辣	1	128.00
1426344746770563073	米饭	44e0e090-b44e-4	1426344746728620034	1423329009705463809	(Null)	(Null)	1	2.00
1426344746770563074	雪碧	f5657d6d-a283-4	1426344746728620034	1423328235936067586	(Null)	(Null)	1	5.00

同时，购物车的数据被删除：

对象	shopping_cart @reggie (localhost) - ...									
开始事务	文本	筛选	排序	导入	导出					
id	name	image	user_id	dish_id	setmeal_id	dish_flavor	number	amount		
(N/A)	(N/A)	(N/A)	(N/A)	(N/A)	(N/A)	(N/A)	(N/A)	(N/A)		